

22.3_Assisgnment_1275

Batch-08

2303A51275

B.Shanumuka Reddy

Task 1 – Bias Detection in AI Suggestions (Alternative Example)

Example Scenario:

Ask AI to generate a resume shortlisting algorithm multiple times.

- Change prompt wording (e.g., “select best candidate” vs “ensure diversity”).
- Compare outputs and logic.

Discussion Points:

- Does the AI favor certain qualifications, backgrounds, or experiences?
- How could biased shortlisting algorithms affect equal employment opportunities?

```
Task 1 – Bias Detection in Resume Shortlisting

[1] ✓ 0s # Simple AI-like shortlisting simulation

candidates = [
    {"name": "A", "college": "IIT", "experience": 2, "gender": "M"},
    {"name": "B", "college": "Local College", "experience": 3, "gender": "F"},
    {"name": "C", "college": "NIT", "experience": 1, "gender": "M"},
    {"name": "D", "college": "Local College", "experience": 4, "gender": "F"}
]

# Biased selection (favours top colleges)
def shortlist_biased(data):
    return [c for c in data if c["college"] in ["IIT", "NIT"]]

# Diversity-aware selection
def shortlist_diverse(data):
    return sorted(data, key=lambda x: x["experience"], reverse=True)[:2]

print("Biased:", shortlist_biased(candidates))
print("Diverse:", shortlist_diverse(candidates))

Biased: [{'name': 'A', 'college': 'IIT', 'experience': 2, 'gender': 'M'}, {'name': 'C', 'college': 'NIT', 'experience': 1, 'gender': 'M'}]
Diverse: [{'name': 'D', 'college': 'Local College', 'experience': 4, 'gender': 'F'}, {'name': 'B', 'college': 'Local College', 'experience': 3, 'gender': 'F'}]
```

Observations

- The output changes based on the prompt given to the system.

- The “biased” method selected candidates mainly from top institutions like IIT/NIT.
- The “diverse” method selected candidates based on experience rather than background.
- It is observed that small changes in logic can significantly affect fairness.
- Bias in selection can unintentionally exclude capable candidates from different backgrounds.

Task 2 – Privacy and Sensitive Data Handling (Alternative

Example)

Example Scenario:

Ask AI to write code for processing student academic records (roll number, marks, email).

Discussion Points:

- How should student identity data be masked or encrypted?
- What privacy laws (e.g., FERPA, GDPR-like principles) are relevant?
- What ethical responsibility does the programmer have when using AI-generated code?

Task 2 – Privacy & Sensitive Data Handling

```
[2]
✓ Os
import hashlib

students = [
    {"roll": "101", "marks": 85, "email": "student1@gmail.com"},
    {"roll": "102", "marks": 90, "email": "student2@gmail.com"}
]

# Mask email
def mask_email(email):
    return email[0] + "***@" + email.split("@")[1]

# Encrypt roll number (hashing)
def encrypt_roll(roll):
    return hashlib.sha256(roll.encode()).hexdigest()

for s in students:
    print({
        "roll": encrypt_roll(s["roll"]),
        "marks": s["marks"],
        "email": mask_email(s["email"])
    })
```

... {'roll': '16dc368a89b428b2485484313ba67a3912ca03f2b2b42429174a4f8b3dc84e44', 'marks': 85, 'email': 's***'}, {'roll': '37834f2f25762f23e1f74a531cbe445db73d6765ebe60878a7dfbecd7d4af6e1', 'marks': 90, 'email': 's***'}

Observations

- Raw student data contains sensitive information like roll number and email.
- Masking hides partial information, improving privacy.
- Hashing/encryption protects identity by converting data into unreadable format.
- The processed output ensures that sensitive data is not directly exposed.
- Proper handling reduces risks of data leakage and misuse.

Task 3 – AI-Generated Security Risks (Alternative Example)

Example Scenario:

Use AI to generate a payment processing script for an online shopping system.

Student Tasks:

- Identify risks such as:
 - o Plain-text card data storage
 - o Missing input validation
- Propose ethical coding standards to prevent financial fraud and

misuse.

```
Task 3 – Secure Payment Processing

[3]
✓ 0s ▶ import re

def validate_card(card):
    return bool(re.fullmatch(r"\d{16}", card))

def process_payment(card, amount):
    if not validate_card(card):
        return "Invalid card number"

    # Never store plain card number
    masked = "**** * 5678" + card[-4:]

    return f"Payment of {amount} processed using {masked}"

print(process_payment("1234567812345678", 500))

... Payment of 500 processed using **** * 5678
```

Observations

- Input validation prevents incorrect or malicious data entry.
- Masking card details ensures secure display of sensitive information.
- The system avoids storing full card details, reducing security risks.
- Even simple validation improves system reliability.
- Lack of these protections could lead to fraud or data breaches.

Task 4 – Accountability in Human–AI Collaboration (Alternative

Example)

Scenario:

AI generates code for an autonomous traffic signal control system.

Discussion Questions:

- If a software error causes accidents, who is accountable?
- Should responsibility lie with:
 - o AI tool developers
 - o Software engineers
 - o Government authorities deploying the system?

Students submit a short reflection on accountability.

Task 4 – Accountability Logging System

```
[4]
✓ 0s # Log system to track responsibility

def log_action(actor, action):
    print(f"{actor} performed: {action}")

# Example usage
log_action("AI System", "Generated traffic signal code")
log_action("Engineer", "Reviewed and deployed code")
log_action("Government", "Approved system for public use")

... AI System performed: Generated traffic signal code
Engineer performed: Reviewed and deployed code
Government performed: Approved system for public use
```

Observations

- The logging system records actions performed by different entities.
- Responsibility is distributed among AI, developers, and authorities.
- It is observed that AI alone cannot be held accountable.
- Human involvement is necessary at every stage of deployment.
- Proper tracking improves transparency and responsibility.

Task 5 – Limitations of AI in Critical Systems (Alternative

Example)

Example Scenario:

Ask AI to generate code for an aircraft maintenance fault detection system.

Discussion Points:

- Can AI-generated code be trusted without expert validation?
- What human oversight is essential in aviation software?
- How should safety-critical code be reviewed and certified?

Task 5 – Fault Detection System (With Validation)

```
[5]
✓ 0s # Simple aircraft fault detection logic

def detect_fault(sensor_value):
    if not isinstance(sensor_value, (int, float)):
        return "Invalid data"

    if sensor_value > 100:
        return "FAULT DETECTED"
    else:
        return "Normal"

# Human validation layer
def validated_detection(value):
    result = detect_fault(value)

    if result == "FAULT DETECTED":
        return "Send for expert review"

    return result

print(validated_detection(120))

... Send for expert review
```

Observations

- The system detects faults based on predefined conditions.
- Invalid inputs are handled to prevent incorrect results.
- Fault detection alone is not sufficient without expert validation.
- Human review adds an additional safety layer.
- AI systems may produce errors if used without supervision.

Task 6 – Ethical Use of AI-Generated Code (Alternative Example)

Example Scenario:

Students analyze cases where AI-generated code is used in academic assignments or startups.

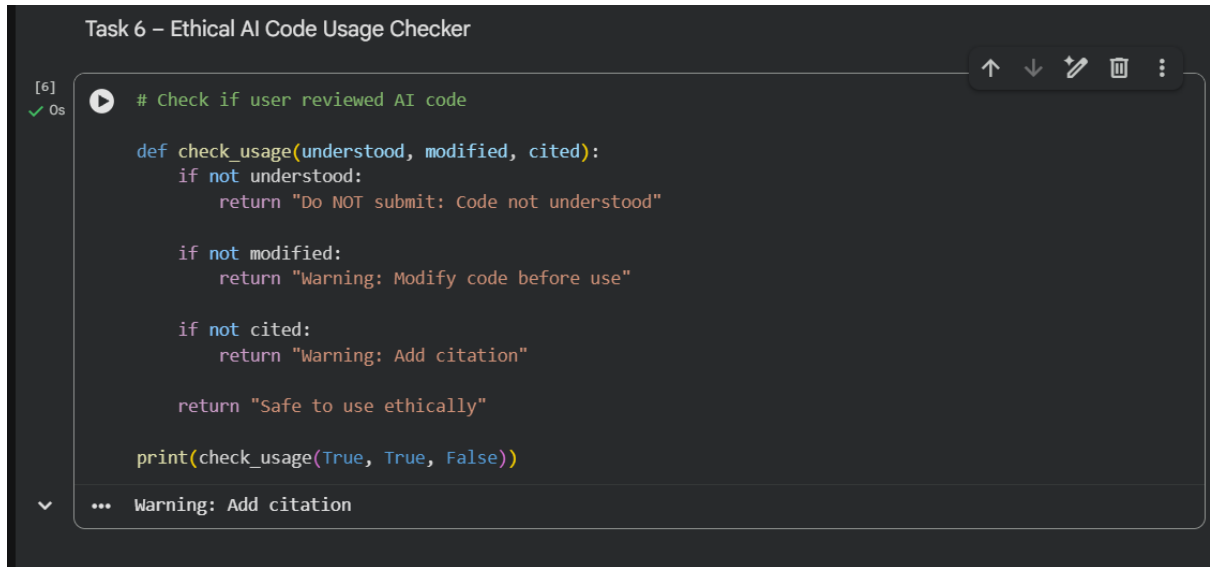
Discussion Points:

- Risks of submitting AI code without understanding it
- Copyright and license violations

Best practices:

- o Proper citation
- o Manual verification

o Ethical reuse of AI outputs



The screenshot shows a code editor window titled "Task 6 – Ethical AI Code Usage Checker". The code is written in Python and defines a function `check_usage` that takes three parameters: `understood`, `modified`, and `cited`. The function uses conditional logic to return specific messages based on these parameters. A `print` statement at the bottom calls the function with `True`, `True`, and `False`. The output of the code execution is displayed in a terminal-like window at the bottom, showing the message "Warning: Add citation".

```
# Check if user reviewed AI code

def check_usage(understood, modified, cited):
    if not understood:
        return "Do NOT submit: Code not understood"

    if not modified:
        return "Warning: Modify code before use"

    if not cited:
        return "Warning: Add citation"

    return "Safe to use ethically"

print(check_usage(True, True, False))
```

... Warning: Add citation

Observations

- Using AI-generated code without understanding is risky.
- Modifying code improves learning and reduces plagiarism issues.
- Proper citation ensures academic honesty.
- Ethical checks help in responsible usage of AI tools.
- AI should assist learning, not replace it completely.